

Constructive Quine Fixed-Point Synthesis, Quadrilateral Filtration, and APG Bisimulation: A Self-Verifiable Foundation for Hypercomputational Reflection

Tyler Roost

Preprint, version 0.1.0 — September 18, 2026

Abstract

We present a formal constructive foundation for self-verifying systems based on three interdependent pillars: a quadrilateral semantic filtration separating execution paths from numerical values, a constructive realization of Kleene-style Quine fixed points, and an explicit quotient representation over Accessible Pointed Graphs (APGs) governed by Aczel’s Anti-Foundation Axiom (AFA). The central impetus of this framework is to discover routes toward definably complete representations capable of handling hypercomputation-like oracle classes expected of superintelligences. By stratifying reflexive self-models through well-founded syntactic strata while accommodating non-well-founded circular data via maximal bisimulation quotients, the system achieves scale-invariant self-closure without collapsing into diagonal paradox, unearned semantic upgrades, or ungrounded truth predicates. We establish the formal correspondence between the underlying algebraic kernel, its constructive Quine synthesizer, and the transfinite ladders of higher-order computational oracles.

Keywords: Hypermath; Quine fixed points; quadrilateral filtration; Accessible Pointed Graphs; Anti-Foundation Axiom; bisimulation quotients; superintelligence oracles.

1 Introduction and Foundational Impetus

The quest for complete or definably complete mathematical representations of intelligent reflection has historically encountered severe limitative results: Gödel’s incompleteness theorems, Tarski’s undefinability of truth, and Turing’s halting problem. When modeling hypothetical superintelligences, these limitations become acute: an advanced cognitive system must be capable of representing, analyzing, and proving properties about its own source code, evaluation traces, and environment models, while interacting with problems requiring computational power equivalent to hypercomputational oracle classes (such as Turing jumps, hyperarithmetic hierarchies, and reflection principles).

If an agent attempts to formalize its own complete truth predicate in an unstratified single-level language, classical diagonal arguments immediately yield inconsistency. If, on the other hand, the system is strictly stratified into an external metalanguage hierarchy, the agent cannot represent its own total operation from within.

Hypermath resolves this apparent impasse through an architecture of *fractal self-representative completeness*:

1. **Quadrilateral Filtration:** Retaining a four-tier distinction between derivation-path identity ($=$), outcome congruence ($\equiv$), continuation similarity ($\sim$), and wrap valuation (W).
2. **Constructive Quine Synthesis:** Explicit algorithmic construction of self-reproducing programs (Quines) that carry their own syntactic codes without circular self-observation.

3. **Non-Well-Founded Graph Semantics:** Representing reflexive and cyclic data structures through Accessible Pointed Graphs (APGs) quotiented under maximal bisimulation, formalizing Aczel's Anti-Foundation Axiom (AFA) constructively.
4. **Stratified Oracle Ladders:** Grounding transfinite hypercomputational reflection in well-founded ordinal ladders (ordinatics) without unearned semantic jumps.

2 The Quadrilateral Filtration

A central design principle of Hypermath is that an evaluation outcome must not erase the computational derivation that produced it. We formalize this through a four-tier relation hierarchy:

Definition 2.1 (The Four Relations). Let $\mathcal{E}$ be the collection of typed syntactic expression trees. We define:

1. **Derivation-path identity** ($=$): Exact syntactic identity of derivation trees, preserving operand order, intermediate steps, and representation choices.
2. **Outcome congruence** ($\equiv$): Equality of denotational values in the target algebraic structure (e.g., identity of polynomials under the representation map j).
3. **Continuation similarity** ($\sim$): Equivalence of downstream operational behaviors under all admissible evaluation contexts.
4. **Wrap valuation** (W): Collapsing evaluation under a designated specialization map (such as $W(X) = -1/2$).

Proposition 2.2 (Strict Invariant Separation). *The four relations satisfy strict inclusion:*

$$(e_1 = e_2) \implies (e_1 \equiv e_2) \implies (e_1 \sim e_2) \implies (W(e_1) = W(e_2)),$$

and none of the reverse implications holds in general.

Proof. Identity of trees implies equality of denoted values. Equality of denoted values ensures identical behavior in every standard algebraic context. Equality under wrap valuation is a coarse homomorphic projection. For the counterexamples:

1. $1 +_o \omega \neq \omega$, but their ordinal value is identical (ω).
2. In ordinal arithmetic, $\omega \cdot_o 2 +_o 1$ and 0 are distinct ordinals, but under polynomial representation $2X + 1$ and specialization at $-1/2$, both yield $W = 0$.

Thus, information is strictly lost upon downward projection. □

3 Constructive Quine Fixed-Point Synthesis

Kleene's Second Recursion Theorem asserts that for any computable transformation F , there exists a program index e such that $\varphi_{F(e)} = \varphi_e$. In a proof system, a Quine is a sentence or term that asserts a property of its own syntactic representation.

Definition 3.1 (Constructive Quine Function). Let $\text{sub}(x, y)$ be the primitive recursive substitution function that substitutes the numeral code of y for the free variable in the expression coded by x . A *Quine constructor* is a pair of terms (u, q) such that

$$q = \text{apply}(u, \ulcorner u \urcorner),$$

where evaluation of q reproduces $\ulcorner q \urcorner$ alongside any specified payload transformation.

Theorem 3.2 (Constructive Self-Closure without Paradox). *Let $\mathcal{L}$ be the ground language generated by constructors $\{\text{Form}, \text{Prop}, \text{ground}, \text{apply}\}$. There exists an effective algorithmic procedure that, given any stage transformation Φ , synthesizes an expression Q_Φ such that*

$$\text{eval}(Q_\Phi) = \langle \ulcorner Q_\Phi \urcorner, \Phi(\ulcorner Q_\Phi \urcorner) \rangle.$$

The evaluation of Q_Φ does not evaluate an unstratified global truth predicate, and its derivation tree is finite and acyclic.

Proof. The synthesis operates via the constructive diagonal map $D(x) = \text{apply}(x, \text{quote}(x))$. Let P be the program that takes input x , computes $D(x)$, and applies Φ to the result: $P(x) = \langle D(x), \Phi(D(x)) \rangle$. Setting $Q_\Phi = D(\ulcorner P \urcorner)$ yields

$$\begin{aligned} Q_\Phi &= \text{apply}(\ulcorner P \urcorner, \text{quote}(\ulcorner P \urcorner)) \\ \implies \text{eval}(Q_\Phi) &= P(\ulcorner P \urcorner) = \langle D(\ulcorner P \urcorner), \Phi(D(\ulcorner P \urcorner)) \rangle = \langle \ulcorner Q_\Phi \urcorner, \Phi(\ulcorner Q_\Phi \urcorner) \rangle. \end{aligned}$$

Since substitution and pairing are primitive recursive, the computation terminates in a bounded number of reduction steps. $\square$

4 Accessible Pointed Graphs and Bisimulation Quotients

To represent circular dependencies, self-referential definitions, and reflexive state transition networks without introducing ill-founded membership trees, Hypermath embeds reflexive objects into Aczel's theory of non-well-founded sets.

Definition 4.1 (Accessible Pointed Graph (APG)). An *Accessible Pointed Graph* is a triple $G = (V, E, r)$ where V is a set of nodes, $E \subseteq V \times V$ is a set of directed edges, and $r \in V$ is a distinguished root node such that every $v \in V$ is reachable from r along a directed path in E .

Definition 4.2 (Bisimulation on APGs). A binary relation $R \subseteq V_1 \times V_2$ between graphs $G_1 = (V_1, E_1, r_1)$ and $G_2 = (V_2, E_2, r_2)$ is a *bisimulation* if $r_1 R r_2$ and for all $u R v$:

1. For every $u' \in V_1$ with $(u, u') \in E_1$, there exists $v' \in V_2$ with $(v, v') \in E_2$ and $u' R v'$.
2. For every $v' \in V_2$ with $(v, v') \in E_2$, there exists $u' \in V_1$ with $(u, u') \in E_1$ and $u' R v'$.

Two APGs are *bisimilar*, written $G_1 \approx G_2$, if there exists a bisimulation between them.

Theorem 4.3 (Aczel's Anti-Foundation and Canonical Quotients). *Under Aczel's Anti-Foundation Axiom (AFA), every APG depicts a unique set. Two APGs depict the same set if and only if they are bisimilar:*

$$\text{set}(G_1) = \text{set}(G_2) \iff G_1 \approx G_2.$$

Furthermore, for every APG G , there exists a unique (up to isomorphism) minimal strongly extensional quotient graph $G/\approx$ obtained by collapsing maximal bisimulation classes.

Proof. This is Aczel's fundamental theorem of hyperset theory [1]. In our constructive setting, for finite APGs, the maximal bisimulation relation is computable in polynomial time via Paige-Tarjan partition refinement [2]. $\square$

Proposition 4.4 (Resolution of Cycle Independence). *A cyclic dependency $x = \{x\}$ depicted by a single self-loop node $G_1 = (\{r\}, \{(r, r)\}, r)$ and a two-cycle graph $G_2 = (\{a, b\}, \{(a, b), (b, a)\}, a)$ satisfy $G_1 \neq G_2$ at the level of derivation paths, but $G_1 \approx G_2$ under bisimulation. Hence they denote the same unique Quine atom Ω in the hyperset quotient.*

5 Transfinite Oracle Ladders and Superintelligence

The constructive Quine synthesis and APG bisimulation quotient establish internal reflexivity for any fixed computational stage. To scale across hypercomputation-like oracle classes expected of superintelligences, Hypermath interfaces directly with transfinite ordinal hierarchies:

1. **Stage 0 (Finite Turing Machines):** Basic reduction and syntax manipulation via the primitive algebraic kernel.
2. **Stage ω (First Jump / Halting Oracle):** Quantifier bounds over arithmetic formulas, reflecting upon the termination of Stage 0 programs.
3. **Stage $\alpha < \omega^\omega$ (Ordinalics Hierarchy):** Stratified satisfaction predicates Tr_α and difference operators Δ^α .
4. **Stage ε_0 and Veblen Ladders (φ_α):** Transfinite fixed-point enumeration, providing exact proof-theoretic bounds for self-reflective reflection principles.

Each higher level serves as an evidence-bearing oracle for lower levels. Because each jump is recorded as an explicit closure coordinate in the Verifier Standard (VSTD) discipline, no agent can claim verification beyond the exact rank and trust root of the discharging mechanism.

6 Machine-Checked Formalization and Its Boundary

The Lean 4 development accompanying this paper [6] compiles under Lean 4.14.0 and depends on no external mathematical library. We separate what the kernel certifies from what remains prose, because that separation is itself a commitment of the framework: a named theorem carries only the weight its proof term earns.

Write $\mathcal{C}$ for the thirty-eight declared logical clauses of the Hypermath kernel, bundled as the Lean structure `FullAxioms`, and write S for the self-closure target `selfDerivationTarget`: the conjunction of ground continuation, reflexivity of D , two-sided identity of composed derivation paths, and the deriver cycle `Simulation(f2f(f2f(deriver)), deriver)`.

Theorem 6.1 (Independence of self-closure from the clause set). *Both $\mathcal{C} \wedge \neg S$ and $\mathcal{C} \wedge S$ are satisfiable. Hence $\mathcal{C} \not\vdash S$ and $\mathcal{C} \not\vdash \neg S$.*

Proof. By two explicit finite models, each checked by the Lean kernel.

(i) `FullAxiomModel.lean` interprets `Forms` as pairs $(n, b) \in \mathbb{N} \times \text{Bool}$ with `Simulation` read as strict syntactic identity. There `full_axioms_hold` establishes $\mathcal{C}$ and `self_derivation_target_fails` establishes $\neg S$; their conjunction is `full_clauses_without_cycle_or_nontrivial_simulation`.

(ii) `FiniteActionCountermodel.lean` interprets `Forms` over six elements, ground lying in a two-cycle and the limit in a disjoint three-cycle. There `full_axioms_hold` again establishes $\mathcal{C}$, while `self_derivation_target_holds` establishes S ; their conjunction is `full_clauses_and_target_without_nonzero_D`. $\square$

The axiom dependencies reported by `#print axioms` are recorded below. No entry depends on `sorryAx`: each rests only on Lean’s standard `propext`, `Classical.choice` and `Quot.sound`, or on nothing at all.

Result	Model	Axiom dependencies
<code>full_axioms_hold</code>	(i)	<code>propext</code> , <code>Classical.choice</code> , <code>Quot.sound</code>
<code>self_derivation_target_fails</code>	(i)	none
<code>full_clauses_without_cycle...</code>	(i)	<code>propext</code> , <code>Classical.choice</code> , <code>Quot.sound</code>
<code>self_derivation_target_holds</code>	(ii)	<code>propext</code>
<code>full_clauses_and_target...</code>	(ii)	<code>propext</code> , <code>Quot.sound</code>

6.1 What is not machine-checked

Theorem 6.1 is the only result in this paper that the Lean kernel certifies. Three limitations follow, and we state them rather than leave them to be discovered.

1. **The constructive Quine synthesis of Section 3 is not formalized.** The scheduled development `lean4/Hypermath/ConstructiveQuine.lean` does not yet exist. Section 3 is a mathematical specification, and the fixed-point construction it describes has not been checked.
2. **The APG bisimulation quotient of Section 4 is not formalized.** The present Lean definitions stipulate bisimulation to be mutual simulation, and the accompanying uniqueness statement reduces to the identity function on that stipulation. It is therefore a tautology and supplies no evidence for the claims of Section 4.
3. **Eleven declarations across the L0–L3 kernel layers use `sorry`.** These are the disclosed proof obligations of the underlying formalization. None of them lies among the clauses of $\mathcal{C}$, and none is a dependency of Theorem 6.1.

The verified contribution is therefore precise, and narrower than the prose above: self-closure is consistent with the current clause set but not entailed by it. It must be *earned* by a stronger semantics, of the kind Sections 3 and 4 propose, rather than assumed or asserted.

7 Conclusion

Hypermath provides an explicit, constructive bridge between finite derivation paths, non-well-founded graph quotients, and transfinite reflection ladders. By distinguishing path identity from bisimulation congruence and wrap valuation, it delivers a rigorous foundation for superintelligent self-models that remain bounded, verifiable, and free of paradox.

References

- [1] Peter Aczel. *Non-Well-Founded Sets*. CSLI Lecture Notes, Vol. 14, Stanford University, 1988.
- [2] Robert Paige and Robert E. Tarjan. Three Partition Refinement Algorithms. *SIAM Journal on Computing* 16(6), 973–989, 1987. [doi:10.1137/0216062](https://doi.org/10.1137/0216062).
- [3] Alfred Tarski. The Semantic Conception of Truth and the Foundations of Semantics. *Philosophy and Phenomenological Research* 4(3), 341–376, 1944.
- [4] Stephen Cole Kleene. *Introduction to Metamathematics*. D. Van Nostrand Co., Inc., New York, 1952.
- [5] Tyler Roost. *Ordinal Arithmetic: An Ordinal-First Metalanguage Approach to Definable Arithmetic Truth*. Preprint, version 0.3.0, 2026.
- [6] Tyler Roost. *Hypermath: a Lean 4 formalization of the ground kernel and its countermodels*. Directory `lean4/`, in particular `FullAxiomModel.lean` and `FiniteActionCountermodel.lean`. github.com/TimeLordRaps/hypermath, 2026.